PUBLIC AUDIT RELEASE

TSwap Security Review

by fuzz755

Table of Contents

About fuzz755

Disclaimer

Risk Classification

Protocol Summary

Executive Summary

- Scope

Findings

- Findings Count
- Findings Summary
- **High Findings**
  - [H-1] DoS and permanent loss of deposits in pools
  - [H-2] Wrong formula leads to fee calculation error and loss of funds for users
  - [H-3] No slippage protection in the `swapExactOutput` function, making it vulnerable to sandwich attacks
  - [H-4] Extra token incentive on swaps is breaking the protocol's invariant
- **Medium Findings**
  - [M-1] The `deadline` parameter is declared but never used in the `deposit` function
  - [M-2] The `_addLiquidityMintAndTransfer` function doesn't follow CEI pattern
  - [M-3] The `sellPoolTokens` function's name is misleading and not working as intended
- **Low Findings**
  - [L-1] The protocol is only handling 18 decimals tokens
- **Informational Findings**
  - [I-1] The variable `MINIMUM_WETH_LIQUIDITY` should use scientific notation
  - [I-2] Use of magic numbers
  - [I-3] Use of magic numbers
- **Gas Findings**
  - [G-1] The custom error `PoolFactory__PoolDoesNotExist` is declared but never used

[G-2] The `poolTokenReserves` local variable is declared but never used in `deposit` function

[G-3] The `swapExactInput` function should be external

[G-4] The `swapExactInput` function always returns zero

About fuzz755

fuzz755 is an independent smart contracts security researcher. Having found numerous security vulnerabilities in various protocols, he does his best to contribute to the blockchain ecosystem and its protocols by putting time and effort into security research & reviews. Check previous work here or reach out on Twitter @fuzz755.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I cannot guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High:** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium:** leads to a moderate material loss of assets in the protocol or moderately harms a group of users.
- **Low:** leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High:** attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium:** only a conditionally incentivized attack vector, but still relatively likely.

- **Low:** has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Protocol Summary

Tswap is a decentralized asset/token exchange (DEX). The protocol is meant to be a permissionless way for users to swap assets between each other at a fair price. It is similar to Uniswap.

Roles

- **Liquidity Providers:** Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- **Users:** Users who want to swap tokens.

Executive Summary

A time-boxed security review of the TSwap repository was conducted by **fuzz755**, from 2026-03-10 — 2026-03-11. A total of **15** issues were uncovered.

Scope

Protocol	TSwap
Protocol Type	AMM / DEX
Audit Date	2026-03-10 — 2026-03-11
Audit Version	1.0
Repository	https://github.com/Cyfrin/5-t-swap-audit
Review Commit Hash	7bd52c6a75f1...

In-Scope Files

`./src/PoolFactory.sol` `./src/TSwapFactory.sol`

Findings

Findings Count

Severity	Count
High	4
Medium	3
Low	1
Informational	3
Gas	4
Total	**15**

Findings Summary

ID	Title	Severity	Status
[H-1]	DoS and permanent loss of deposits in pools	High	Pending
[H-2]	Wrong formula leads to fee calculation error and loss of funds for users	High	Pending
[H-3]	No slippage protection in the `swapExactOutput` function, making it vulnerable to sandwich attacks	High	Pending
[H-4]	Extra token incentive on swaps is breaking the protocol's invariant	High	Pending
[M-1]	The `deadline` parameter is declared but never used in the `deposit` function	Medium	Pending
[M-2]	The `_addLiquidityMintAndTransfer` function doesn't follow CEI pattern	Medium	Pending
[M-3]	The `sellPoolTokens` function's name is misleading and not working as intended	Medium	Pending
[L-1]	The protocol is only handling 18 decimals tokens	Low	Pending
[I-1]	The variable `MINIMUM_WETH_LIQUIDITY` should use scientific notation	Informational	Pending

ID	Title	Severity	Status
[I-2]	Use of magic numbers	Informational	Pending
[I-3]	Use of magic numbers	Informational	Pending
[G-1]	The custom error `PoolFactory__PoolDoesNotExist` is declared but never used	Gas	Pending
[G-2]	The `poolTokenReserves` local variable is declared but never used in `deposit` function	Gas	Pending
[G-3]	The `swapExactInput` function should be external	Gas	Pending
[G-4]	The `swapExactInput` function always returns zero	Gas	Pending

High Findings

[H-1] DoS and permanent loss of deposits in pools

HIGH Pending

DESCRIPTION

When a user calls the `TSwapPool::deposit` function, they can use the parameter `maximumPoolTokensToDeposit` to chose how many pool tokens they will deposit at most (corresponding to the amount of WETH). When this happens, we have 2 possible cases:

1. The pool already has liquidity: in this case, the `maximumPoolTokensToDeposit` variable is protecting the user from sending too much pool tokens to the pool.
2. The pool has no liquidity: in this case, the `maximumPoolTokensToDeposit` variable is used to define the initial amount of pool tokens inside of the pool. The issue is related to the second case. A user may set the parameter to zero, and send a minimum amount of WETH in the pool. By doing so, the pool will have `X=0` pool tokens and `Y` WETH. This means that the pool token has an infinite price and we will probably have a zero division somewhere. But there is a simpler problem: users will never be able to withdraw from a pool with `X=0` pool tokens. Why? Look at this snippet from the `withdraw` function:

```
if (poolTokensToWithdraw < minPoolTokensToWithdraw) {
    revert TSwapPool__OutputTooLow(
        poolTokensToWithdraw,
        minPoolTokensToWithdraw
    );
}
```

The function is taking a `minPoolTokensToWithdraw` parameter to let users chose how many pool tokens they except to receive. If the pool has 0 pool tokens to withdraw, we will be in a situation where the condition `poolTokensToWithdraw < minPoolTokensToWithdraw` is always true except when `minPoolTokensToWithdraw == 0`. However the function is doing a zero check:

```
revertIfZero(minPoolTokensToWithdraw)
```

If the `minPoolTokensToWithdraw` parameter is set to `0`, the transaction will revert because of this.

IMPACT

Liquidity providers' WETH are permanently lost, and the pool is not usable.

RECOMMENDED MITIGATION

Do a zero check on deposits:

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
        revertIfZero(wethToDeposit)
+       revertIfZero(maximumPoolTokensToDeposit)
        returns (uint256 liquidityTokensToMint)
```

[H-2] Wrong formula leads to fee calculation error and loss of funds for users

HIGH | Pending

DESCRIPTION

The `getInputAmountBasedOnOutput` function is used in `swapExactOutput` to calculate the amount of input tokens to pay in order to get a certain amount of output tokens. However, the return formula is wrong, it is multiplying the final output by `10000` instead of `1000`. Resulting in a return value 10 times higher than it should be.

IMPACT

Fees are 90.03% instead of 0.3%. Swappers will pay way more fees than expected to liquidity providers.

RECOMMENDED MITIGATION

Fix the formula:

```
    return
-        ((inputReserves * outputAmount) * 10000) /
+        ((inputReserves * outputAmount) * 1000) /
        ((outputReserves - outputAmount) * 997);
```

[H-3] No slippage protection in the `swapExactOutput` function, making it vulnerable to sandwich attacks

HIGH Pending

DESCRIPTION

Slippage protection is important to protect users from trading at unpredictable prices. This is usually a medium issue, but in our case, combined with the [H-2] issue it is a critical vulnerability. The price formula is wrong when calculating the input amount with `getInputAmountBasedOnOutput`. The input amount will be way higher than it should be. Without slippage protection, the transaction will pass if the user has enough tokens and approval. This also exposes users to sandwich attacks.

IMPACT

Users will pay way more than expected for a swap.

PROOF OF CONCEPT

Actors:

- Attacker: A liquidity provider performing a sandwich attack on User
- User: Normal user

Description:

1. The attacker owns the whole liquidity pool assets
2. The Attacker waits for a user to swap with the `swapExactOutput` function.
3. The Attacker buys pool tokens just before the User
4. The User buys pool tokens and pays a lot of WETH
5. The Attacker sells its pool tokens with sandwich attack profit + wrong fee formula profits

Proof of Code:

```
function test_sandwichAttack() public {
    // ADD LIQUIDITY

    vm.startPrank(liquidityProvider);

    poolToken.mint(liquidityProvider, 100e18);
    weth.mint(liquidityProvider, 10e18);

    poolToken.approve(address(pool), 100e18);
    weth.approve(address(pool), 10e18);
```

```
        pool.deposit(
            10e18,
            1e18,
            100e18,
            uint64(block.timestamp + 30)
        );

        // SANDWICH BUY

        uint256 inputAmount = pool.getInputAmountBasedOnOutput(
            1e17,
            weth.balanceOf(address(pool)),
            poolToken.balanceOf(address(pool))
        );
        weth.mint(liquidityProvider, inputAmount);
        weth.approve(address(pool), inputAmount);

        uint256 expectedOutputAmount = pool.getOutputAmountBasedOnInput(
            inputAmount,
            weth.balanceOf(address(pool)),
            poolToken.balanceOf(address(pool))
        );
        console2.log(expectedOutputAmount);

        pool.swapExactOutput(
            weth,
            poolToken,
            1e17,
            uint64(block.timestamp + 30)
        );

        vm.stopPrank();

        // USER IS BUYING

        vm.startPrank(user);

        inputAmount = pool.getInputAmountBasedOnOutput(
            1e16,
            weth.balanceOf(address(pool)),
            poolToken.balanceOf(address(pool))
        );
        weth.mint(user, inputAmount);
        weth.approve(address(pool), inputAmount);

        uint256 actualOutputAmount = pool.getOutputAmountBasedOnInput(
            inputAmount,
            weth.balanceOf(address(pool)),
            poolToken.balanceOf(address(pool))
```

```
        );
        console2.log(actualOutputAmount);

        pool.swapExactOutput(
            weth,
            poolToken,
            1e16,
            uint64(block.timestamp + 30)
        );

        vm.stopPrank();

        // SANDWICH SELL

        vm.startPrank(liquidityProvider);

        inputAmount = pool.getInputAmountBasedOnOutput(
            1e17,
            weth.balanceOf(address(pool)),
            poolToken.balanceOf(address(pool))
        );
        weth.mint(liquidityProvider, inputAmount);
        weth.approve(address(pool), inputAmount);

        pool.swapExactOutput(
            weth,
            poolToken,
            1e17,
            uint64(block.timestamp + 30)
        );

        vm.stopPrank();

        assertGt(expectedOutputAmount, actualOutputAmount);
    }
```

RECOMMENDED MITIGATION

Add a `maxInputAmount` parameter to the `swapExactOutput` function:

```
+    error TSwapPool__InputTooHigh(uint256 actual, uint256 max);

    ...

    function swapExactOutput(
        IERC20 inputToken,
+        uint256 maxInputAmount
```

```
        IERC20 outputToken,
        uint256 outputAmount,
        uint64 deadline
    )
        public
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

        inputAmount = getInputAmountBasedOnOutput(
            outputAmount,
            inputReserves,
            outputReserves
        );

+        if (inputAmount > maxInputAmount) {
+            revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount);
+        }

        _swap(inputToken, inputAmount, outputToken, outputAmount);
    }
```

[H-4] Extra token incentive on swaps is breaking the protocol's invariant

HIGH Pending

DESCRIPTION

Every 10 swaps, the protocol is giving the swap caller an extra token as an extra incentive to keep trading on T-Swap. However, this behavior is breaking the protocol's invariant, as specified in the docs:

```
Invariant with fees
ρ = fee (between 0 & 1, aka a percentage)
γ = (1 - ρ) (pronounced gamma)
Δx = (β/(1-β)) * (1/γ) * x
Δy = (αγ/1+αγ) * y
```

IMPACT

This will lead to protocol insolvency.

PROOF OF CONCEPT

Check `test/Invariant.t.sol`.

RECOMMENDED MITIGATION

Remove this feature or use the fees eventually generated by the protocol.

Medium Findings

[M-1] The `deadline` parameter is declared but never used in the `deposit` function

MEDIUM Pending

DESCRIPTION

Using a deadline on swaps is a safety precaution to avoid trades to be executed at unpredicatable price changes. If the deadline is reached without the swap being executed, the transaction should revert and let the user choose if he wants to try again at the new price. The `deadline` is declared as a parameter inside the `deposit` function but it is never used.

IMPACT

Unfavorable prices for users.

RECOMMENDED MITIGATION

Use the deadline check modifier:

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
    revertIfZero(wethToDeposit)
+    revertIfDeadlinePassed(deadline)
    returns (uint256 liquidityTokensToMint)
```

[M-2] The `_addLiquidityMintAndTransfer` function doesn't follow CEI pattern

MEDIUM Pending

DESCRIPTION

When the `_addLiquidityMintAndTransfer` function is called, the user is receiving newly minted liquidity tokens before sending the assets to the pool. This is not complying with CEI pattern which can lead to unexpected behaviors with ERC-777 tokens that allow reentrency.

IMPACT

Reentrency risk.

RECOMMENDED MITIGATION

Follow CEI pattern

```
    ) private {
-       _mint(msg.sender, liquidityTokensToMint);
-       emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);

        // Interactions
        i_wethToken.safeTransferFrom(msg.sender, address(this), wethToDeposit);
        i_poolToken.safeTransferFrom(
            msg.sender,
            address(this),
            poolTokensToDeposit
        );

+       _mint(msg.sender, liquidityTokensToMint);
+       emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
    }
```

[M-3] The `sellPoolTokens` function's name is misleading and not working as intended

MEDIUM | Pending

DESCRIPTION

The wrapper function `sellPoolTokens` has been created to be a helper function, that will let user sell pool tokens into WETH with a single parameter `poolTokenAmount`. If the user wants to sell tokens, the input should be the pool token and the output should be WETH. However, the `sellPoolTokens` function is calling this function:

```
    swapExactOutput(
        i_poolToken,
        i_wethToken,
        poolTokenAmount,
        uint64(block.timestamp)
    );
```

This function allows users to swap by choosing the amount of output tokens to receive, but it is taking `poolTokenAmount` as a parameter, which is our input token. If we want to use the amount of pool tokens as a parameter, we need to use `swapExactInput` instead.

IMPACT

Users will sell the wrong amount of pool tokens if they call the function.

RECOMMENDED MITIGATION

Change the function called inside of `sellPoolTokens`:

```
function sellPoolTokens(
    uint256 poolTokenAmount
) external returns (uint256 wethAmount) {
    return
        swapExactInput(
            i_poolToken,
            poolTokenAmount,
            i_wethToken,
            0
            uint64(block.timestamp)
        );
}
```

Low Findings

[L-1] The protocol is only handling 18 decimals tokens

LOW Pending

DESCRIPTION

If a pool is created for a token with less or more than 18 decimals, the 2 price getter functions `getPriceOfOneWethInPoolTokens` and `getPriceOfOnePoolTokenInWeth` will return wrong values.

RECOMMENDED MITIGATION

Use the decimals function of the ERC-20 interface.

Informational Findings

[I-1] The variable `MINIMUM_WETH_LIQUIDITY` should use scientific notation

INFO Pending

DESCRIPTION

Scientific notation makes it easier to read big numbers, Ethereum is using 1e18 decimals and 1_000_000_000 can be written as 1e9.

RECOMMENDED MITIGATION

Use scientific notation:

```
-    uint256 private constant MINIMUM_WETH_LIQUIDITY = 1_000_000_000;
+    uint256 private constant MINIMUM_WETH_LIQUIDITY = 1e9;
```

[I-2] Use of magic numbers

INFO Pending

DESCRIPTION

The `getOutputAmountBasedOnInput` function is calculating a local variable using magic numbers to process fees: `997` and `1000`. These numbers should be constants and have a proper name describing what they do.

RECOMMENDED MITIGATION

Create constant variables in the contract storage:

```
     uint256 private constant MINIMUM_WETH_LIQUIDITY = 1_000_000_000;
     uint256 private swap_count = 0;
     uint256 private constant SWAP_COUNT_MAX = 10;
+     // divide FEE by FEE_PRECISION to get the true fee ratio
+     uint256 private constant FEE = 997;
+     uint256 private constant FEE_PRECISION = 1000;

...

-    uint256 inputAmountMinusFee = inputAmount * 997;
```

```
+    uint256 inputAmountMinusFee = inputAmount * FEE;
     uint256 numerator = inputAmountMinusFee * outputReserves;
-    uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;
+    uint256 denominator = (inputReserves * FEE_PRECISION) + inputAmountMinusFee;
     return numerator / denominator;
```

[I-3] Use of magic numbers

INFO | Pending

DESCRIPTION

The `getInputAmountBasedOnOutput` function is calculating a local variable using magic numbers to process fees: `997` and `10000`. These numbers should be constants and have a proper name describing what they do.

IMPACT

Using magic numbers is often leading to mistakes, just like here: it should be `1000` not `10000`.

RECOMMENDED MITIGATION

Create constant variables in the contract storage:

```
     uint256 private constant MINIMUM_WETH_LIQUIDITY = 1_000_000_000;
     uint256 private swap_count = 0;
     uint256 private constant SWAP_COUNT_MAX = 10;
+    // divide FEE by FEE_PRECISION to get the true fee ratio
+    uint256 private constant FEE = 997;
+    uint256 private constant FEE_PRECISION = 1000;

...

-    ((inputReserves * outputAmount) * 10000) /
-    ((outputReserves - outputAmount) * 997);
+    ((inputReserves * outputAmount) * FEE_PRECISION) /
+    ((outputReserves - outputAmount) * FEE);
```

Gas Findings

[G-1] The custom error `PoolFactory__PoolDoesNotExist` is declared but never used

GAS Pending

DESCRIPTION

This error is intended to be used inside of the `getToken` function, which is external. Using errors on external function is not useful. The error is not used anyway so it is just wasting gas.

RECOMMENDED MITIGATION

Delete the error:

```
-    error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[G-2] The `poolTokenReserves` local variable is declared but never used in `deposit` function

GAS Pending

DESCRIPTION

This variable is useless and was probably created for documentation. It can be explained in comments to avoid consuming gas for no reason.

RECOMMENDED MITIGATION

Delete the variable:

```
-    uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

[G-3] The `swapExactInput` function should be external

GAS Pending

DESCRIPTION

A public function that is never used inside of a smart contract can be external to consume less fees.

RECOMMENDED MITIGATION

Replace `public` by `external`:

```
    function swapExactInput(
        IERC20 inputToken,
        uint256 inputAmount,
        IERC20 outputToken,
        uint256 minOutputAmount,
        uint64 deadline
    )
-        public
+        external
        revertIfZero(inputAmount)
        revertIfDeadlinePassed(deadline)
```

[G-4] The `swapExactInput` function always returns zero

GAS | Pending

DESCRIPTION

The `swapExactInput` function is returning a variable named `output`. However, this variable is never used inside the function. Because of this, the function will always return zero. The intended behavior is probably to return the output amount which is named `outputAmount`.

RECOMMENDED MITIGATION

Rename the return variable:

```
    function swapExactInput(
        IERC20 inputToken,
        uint256 inputAmount,
        IERC20 outputToken,
        uint256 minOutputAmount,
        uint64 deadline
    )
```

```
        public
        revertIfZero(inputAmount)
        revertIfDeadlinePassed(deadline)
-        returns (uint256 output)
+        returns (uint256 outputAmount)
    {

...

-        uint256 outputAmount = getOutputAmountBasedOnInput(
+        outputAmount = getOutputAmountBasedOnInput(
            inputAmount,
            inputReserves,
            outputReserves
        );
```